

3. Phase 1 UBL Syntax Binding: Environment, Conversion Operations, and Function-Level Processing

This document covers **src/syntax_binding.py**: Phase 1 input definitions, syntax-binding tables, forward conversion, reverse conversion, xBRL-CSV metadata, validation, and function-level processing.

Processing Flow

```
UBL Invoice XML
→ load binding and LHM layout
→ evaluate XPath/selectors
→ resolve dimension ownership
→ emit sparse Structured CSV rows
→ generate xBRL-CSV metadata
→ (reverse) reconstruct and schema-order UBL XML
```

Complete Phase 1 Contract and User Guide

1. Purpose

This document specifies the syntax binding conversion program used by the UADC Proof of Concept.

The program converts an XML invoice document into a dimension-based hierarchical CSV by applying a syntax binding CSV. During forward conversion it also writes JSON metadata that relates the CSV columns to the generated taxonomy. It can also reverse a hierarchical CSV back to XML with the same binding information.

Implementation-level processing details are included in Chapters 7 through 10 and Chapter 15 of this document.

All paths in this document are relative to the **UADC_PoC** working directory after the repository is pushed or cloned, except paths beginning with **../**, which refer to sibling directories of **UADC_PoC**.

2. Main Program

Program:

```
src/syntax_binding.py
```

The converter is self-contained for the supported namespace and XPath helper functions used by the operational hierarchical converter.

3. Input Files

3.1 XML input

The XML input is a UBL Invoice XML document.

PoC sample:

```
samples/input/openpeppol_ubl_invoice_minimal.xml
```

Package sample:

```
../syntax_binding_revised_package/invoice.xml
```

3.2 Syntax binding CSV

The binding CSV maps semantic model paths to XML XPath expressions.

PoC binding:

```
specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv
```

Package binding:

```
../syntax_binding_revised_package/bindings.csv
```

Required operational binding columns:

```
sequence  
level  
structured_csv_level  
type  
name  
datatype  
multiplicity  
semantic_path  
structured_csv_column
```

```
element
xpath
```

Rows without both **semantic_path** and **xpath** are ignored for fact extraction. **type=C** rows define semantic class contexts and repeated row scopes. **type=A** rows define extractable or generatable values.

3.3 Binding-Embedded LHM Layout

The syntax binding CSV is defined by copying the required LHM columns and preserving the LHM row order. The converter therefore uses the binding table itself to define:

- output column order;
- BG dimension columns;
- BT value columns;
- mapping from semantic paths to final LHM **element** names;
- repeating BGs based on **multiplicity**;
- repeat context XPath for BG rows where available.

The LHM source remains the governance source for these columns:

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
```

3.4 Template CSV

When **--template-csv** is supplied, the converter uses its header as the output column order and infers some field-to-dimension placement from non-empty template rows.

Package template:

```
../syntax_binding_revised_package/invoice.csv
```

If **--template-csv** is supplied, the binding-derived layout controls the main field order and the template may still provide field placement hints.

4. Forward Output

The output is a UTF-8-SIG CSV file.

PoC output:

```
out/phase1/openpeppol_ubl_invoice_minimal.csv
```

Package output:

```
out/phase1/<input-xml-stem>.csv
```

The output contains:

- **dInvoice** as the root dimension column;
- additional **d*** columns for repeating BGs;
- BT value columns after dimension columns;
- one root row for invoice-level facts;
- one row per repeated BG context where bindings produce values.

Forward conversion also writes JSON metadata. By default the metadata file name is:

```
<output-csv-stem>.json
```

For example:

```
out/phase1/openpeppol_ubl_invoice_minimal.json
```

The metadata is an xBRL-CSV metadata document that Arelle can load with the **loadFromOIM** plugin. It links the structured CSV table to the generated xBRL-CSV taxonomy entry points and maps CSV columns to taxonomy concepts. The report **documentInfo.taxonomy** property must not be empty; missing **en16931-oim** is treated as a conversion error. The JSON metadata names the xBRL-CSV OIM taxonomy entry point **out/taxonomy/plt/en16931-oim-.xsd**.

5. Reverse Output

When **--reverse** is used, the input is a hierarchical CSV and the output is an XML file.

Reverse output example:

```
out/reverse/en16931_reverse_invoice.xml
```

The reverse converter:

- reads the hierarchical CSV rows;
- resolves fields through the syntax binding CSV;
- uses the binding-derived layout to associate fields with dimension rows;
- creates UBL Invoice XML elements from bound XPath expressions;
- creates one XML context for each repeated dimension row that has bound values.

- derives required **currencyID** attributes for amount elements from invoice currency fields.
- keeps out-of-context absolute XPaths rooted at the document even when the semantic term belongs to a repeated class;
- writes BT-90 below **AccountingSupplierParty** and never creates a nested **Invoice** below **PaymentMeans**.

6. Command-Line Interface

```
syntax_binding.py XML_FILE -b BINDING_CSV -o OUTPUT_CSV [options]
```

Reverse form:

```
syntax_binding.py INPUT_CSV --reverse -b BINDING_CSV -o OUTPUT_XML  
[options]
```

Arguments:

- **XML_FILE**: input XML file in forward mode.
- **INPUT_CSV**: input hierarchical CSV file in reverse mode.
- **-b, --binding**: syntax binding CSV.
- **-o, --output**: output hierarchical CSV in forward mode, or output XML in reverse mode.
- **--template-csv**: optional CSV template defining column order and dimension placement.
- **--metadata-output**: optional JSON metadata output path. Default is the output CSV path with **.json** extension.
- **--taxonomy-base**: generated taxonomy base directory referenced by JSON metadata. Default is **out/taxonomy**.
- **--reverse**: convert hierarchical CSV back to XML.
- **--drop-empty-columns**: remove columns that have no values in the generated rows.
- **--d-invoice**: value written into the **dInvoice** column. Default is **1**.
- **-e, --encoding**: CSV encoding. Default is **utf-8-sig**.

7. Forward Processing Model

7.1 Namespace handling

The converter collects XML namespace declarations from the input XML using **xml.etree.ElementTree.iterparse**.

XPath helper functions support namespace-prefixed elements such as:

```
/Invoice/cbc:ID  
/Invoice/cac:AccountingSupplierParty/cac:Party/cac:PartyName/cbc:Name
```

7.2 Binding-Embedded LHM Layout Loading

When the syntax binding table is loaded:

1. C rows are treated as BG/class rows.
2. A C row becomes a dimension only if it is the invoice root or if its multiplicity is repeating, such as **0..*** or **1..***.
3. Non-repeating BG rows remain semantic grouping nodes and are not emitted as dimensions.
4. Dimension names are generated as **d** + UpperCamelCase(**element**).
5. A rows are treated as BT/fact rows.
6. Fact column names are taken from **structured_csv_column**.
7. Each fact is assigned to the nearest ancestor repeated dimension.

Repeating multiplicity is detected when the upper bound is:

```
*
n
unbounded
```

or a numeric value greater than **1**.

7.3 Binding parsing

Each binding row is converted into an internal binding record:

```
semantic_path
xpath
root
dimension
filter_field
filter_value
field
```

The converter supports:

- root-level paths such as **\$.invoice.invoiceNumber**;
- dimension paths with optional filter predicates;
- generic nested semantic paths resolved through the nearest LHM dimension.

7.4 Row generation

Forward conversion uses XML parent context recursion rather than processing repeated groups independently. The converter creates:

- a root invoice row for invoice-level facts;

- repeated BG rows for repeated contexts such as VAT breakdown or invoice line.

For repeated BG rows:

1. The converter builds a **BindingClass** tree from **type=C** rows.
2. The root call starts with **dInvoice=1**.
3. For the current class, direct **type=A** facts are extracted with XPath relative to the current XML context.
4. Non-repeated child classes reuse the current Structured CSV row.
5. Repeated child classes are processed one occurrence at a time under the current parent XML context.
6. Each repeated occurrence receives a 1-based dimension value, such as **dInvoiceLine=2**.
7. Ancestor repeated dimension values are copied into nested repeated rows.

A non-repeating child class is flattened into its parent row. A repeating child class is not flattened: its first and subsequent occurrences are all written on separate child rows. For example:

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

The parent row therefore has empty **b1**, **b2**, and **b3** fields. A row such as **1,1,a1V1,a2V1,b1V1,b2V1,b3V1** is invalid because it mixes facts owned by **dAaa** with facts owned by the repeating **dBbb** scope.

7.5 Column handling

If the LHM is used, field order is:

```
dimension columns first, then fact columns
```

If a template is used without LHM, the template header controls the output order.

If neither LHM nor template provides a header, field names are derived from bindings.

8. Metadata JSON Processing Model

In forward mode the converter writes JSON metadata after the structured CSV is generated.

The metadata structure follows the xBRL-CSV report package shape:

- **documentInfo**: xBRL-CSV document type, namespaces, and taxonomy entry point references.
- **tables**: the generated structured CSV table and its relative URL.
- **tableTemplates**: table-level dimensions and per-column concept mappings.

Dimension columns are declared as table dimensions:

```
"plt:d_en16931_Invoice": "$dInvoice"
```

Fact columns are declared with xBRL concepts:

Example:

```
{
  "InvoiceNumber": {
    "dimensions": {
      "concept": "en16931:InvoiceNumber"
    }
  }
}
```

Amount facts receive a default test unit of **iso4217:JPY** in metadata when the LHM datatype indicates an amount. Production use should derive the unit from the applicable currency term.

9. Reverse Processing Model

In reverse mode:

1. The converter reads the input hierarchical CSV with **csv.DictReader**.
2. The converter reads the binding-derived layout and binding CSV in the same way as forward mode.
3. Before XML creation, each fact is checked against the deepest populated dimension on its row. Facts owned by an ancestor dimension are rejected on a repeated child row, so a mixed parent/child row cannot be reverse-converted.
4. Root-level fields are written to XML paths directly below the invoice root.
5. Dimension fields are grouped by their **d*** dimension column.
6. For each dimension row with bound values, the converter creates a repeated XML context from the BG XPath.
7. Bound field values are written to relative XPaths under that context.
8. XPath predicates such as **cbc:ChargeIndicator=false()** and **cbc:DocumentTypeCode='130'** are materialized as child values when a matching XML context is created.
9. Amount elements receive **currencyID** from the invoice document currency field. Tax accounting currency TaxTotal paths receive **currencyID** from the tax accounting currency field.
10. If **relative_xpath(binding.xpath, repeat_path)** remains absolute, the binding is outside the repeated syntax context and is written from the document root. Only contained paths are written relative to the repeated element.
11. UBL-required supporting elements that are not EN 16931 BT values are added where needed for schema validation, including **cac:TaxScheme/cbc:ID** and missing

cbc:ChargeIndicator=false.

12. Generated UBL child elements are normalized to UBL schema sequence. When **--ubl-schema-root** or **--ubl-schema-url** is supplied, the converter derives child order from XSD **xs:sequence** declarations. Without a schema source, it uses the built-in fallback order for the supported Invoice structures.

The reverse converter currently targets the PoC UBL Invoice binding pattern. It is intended for round-trip verification of the structured CSV representation rather than canonical XML reproduction.

10. Currency Attribute Rules

The syntax binding treats **currencyID** as syntax metadata derived from semantic currency terms:

- BT-5 **DocumentCurrencyCode** is the default **currencyID** for invoice amount elements.
- BT-6 **TaxAccountingCurrencyCode** is the **currencyID** for the tax accounting currency TaxTotal branch.
- BT-110 and BG-23 use the UBL path predicate **cbc:TaxAmount/@currencyID=/Invoice/cbc:DocumentCurrencyCode**.
- BT-111 uses the UBL path predicate **cbc:TaxAmount/@currencyID=/Invoice/cbc:TaxCurrencyCode**.

Forward conversion evaluates these absolute predicate reference paths from the document root. For **Allowance-example.xml**, BT-110 resolves to **1225.00 EUR** and BT-111 resolves to **9324.00 SEK**.

BT-90 illustrates a cross-scope reverse mapping. Its semantic path is under payment instructions/direct debit, but its UBL XPath points to **/Invoice/cac:AccountingSupplierParty/.../cbc:ID**. Reverse conversion therefore keeps that XPath document-rooted rather than constructing **PaymentMeans/Invoice/AccountingSupplierParty**.

The CSV therefore stores semantic amount values and currency code values separately. Reverse conversion writes the required UBL **currencyID** attributes.

11. Constraints

- The converter does not validate the XML against UBL schemas internally; the regression test **tests/test_roundtrip_xml_ubl_schema.py** performs UBL 2.1 schema validation.
- The converter does not validate EN 16931 business rules.
- The converter does not run Arelle internally; the regression test **tests/test_xbrl_csv_metadata_arelle.py** validates generated metadata with Arelle.
- The converter does not evaluate full XPath 2.0.
- Supported predicates are intentionally limited to the cases used by the PoC bindings.
- Template columns alone do not create rows; rows require matching bindings.
- Reverse XML element order can be derived from a supplied UBL XSD schema root or UBL Invoice schema URL. The built-in child-order table in **syntax_binding.py** is only the fallback when no schema source is supplied.
- Reverse XML may omit unbound XML content.

- Reverse XML is generated from bound CSV values and is not intended to be byte-for-byte identical to the source XML.

12. Regression Tests

PoC LHM-driven conversion test:

```
tests/test_lhm_hierarchical_csv_layout.py
```

Package template/binding conversion test:

```
tests/test_syntax_binding.py
```

OpenPeppol structured conversion test:

```
tests/test_openpeppol_invoice_conversion.py
```

Reverse conversion round-trip test:

```
tests/test_syntax_binding_reverse.py
```

Round-trip artifact and metadata test:

```
tests/test_roundtrip_artifacts.py
```

13. Non-Goals

The syntax binding converter does not:

- generate LHM definitions;
- generate XBRL taxonomy files;
- run semantic binding to flat CSV;
- perform full EN 16931 or OpenPeppol validation;
- perform repository synchronization or publication.

14. Operational Workflow

14.1 Prepare Phase 1 Inputs

Place source UBL invoices in the configured input directory and select the matching syntax-binding CSV. Binding rows with **type=C** define the class tree; rows with **type=A** define fact extraction and reverse-writing rules. Verify the binding version before processing because the table is runtime authority.

14.2 Convert and Inspect

Run **src/syntax_binding.py** for one XML document or an input directory. Inspect the generated Structured CSV for sparse parent and repeated-child rows, and inspect the JSON metadata for taxonomy entry points, dimensions, and units. Chapter 4 of **02_STRUCTURED_CSV_LHM_BINDINGS.md** defines the required row pattern.

14.3 Reverse and Validate

Use reverse mode to reconstruct UBL. The converter starts from the document root, creates absolute XPath ancestors in schema order, applies predicates and currency attributes, and omits empty decimal elements. Validate the resulting XML with the UBL schema and compare a second forward conversion with the original Structured CSV.

15. Function-Level Processing Reference

15.1 Binding and Layout Loading

- **read_binding_rows** reads the binding CSV without losing row order.
- **build_layout_from_rows** derives columns, dimension ownership, class multiplicity, and binding-embedded LHM layout.
- **read_bindings** converts attribute rows into binding objects and resolves semantic paths.
- **build_binding_class_tree** builds the nested class model used by both directions.
- **direct_class_fields** and **walk_binding_classes** associate facts with their owning class and traverse it deterministically.

15.2 XPath Evaluation

- **collect_namespaces** records namespaces declared by the input XML.
- **xml_split_xpath**, **xml_split_step_predicate**, and **xml_split_terminal_attribute** decompose binding XPath safely.
- **find_nodes**, **xml_predicate_matches**, and **get_value** evaluate XPath relative to the current class context while allowing document-root references in predicates.
- **infer_repeat_path**, **common_xpath_prefix**, and **relative_xpath** derive the XML node scope for repeated classes.

15.3 Forward Row Emission

- **write_hierarchical_csv** orchestrates recursive XML traversal, dimension numbering, sparse row emission, column ordering, and metadata creation.
- **new_row** creates a row with its ancestor dimensions.

- **row_has_values** prevents structurally empty output rows.
- **validate_hierarchical_row_scopes** rejects a repeated child's first values when they have been merged into the parent row.
- **drop_empty_columns** removes unused value columns while preserving required dimensions.

For a singular child, extracted fields remain on the nearest repeated ancestor row. For a repeated child, the parent row is emitted separately and every child, including occurrence 1, is emitted on its own dimension row.

15.4 Metadata Generation

- **binding_column_metadata** derives concept, datatype, dimension, and unit information from the binding.
- **taxonomy_entrpoints** selects the dated taxonomy entry point.
- **xbml_csv_column_definition** creates OIM-compatible column definitions.
- **write_csv_metadata** writes JSON metadata and relative table paths.

15.5 Reverse XML Construction

- **write_xml_from_hierarchical_csv** groups rows by dimension scope and drives XML reconstruction.
- **split_xml_path**, **ensure_path**, and **find_or_create_child** process the absolute XPath from the UBL root; they never search globally for a matching descendant.
- **set_xml_value**, **set_relative_xml_value**, and **set_xml_value_with_currency** write values only when a non-empty source value exists.
- **apply_currency_attribute** and **resolve_currency_references** distinguish document currency from tax accounting currency for conditional amounts.
- **load_ubl_child_order** and **sort_children_for_ubl_schema** place elements in UBL schema sequence before serialization.

15.6 Validation Boundary

The converter validates its hierarchical row contract and reconstructs schema-shaped XML, but external UBL schema validation remains a required test step. Business-rule validation such as EN 16931 and Peppol Schematron is outside the converter and should be run by the surrounding workflow.

16. Syntax Binding Conversion Overview

Syntax Binding Conversion Documentation

This directory documents the XML-to-structured-CSV and structured-CSV-to-XML conversion program.

For the detailed implementation-level explanation of XPath context processing, Semantic Path resolution, **dInvoice** and **dInvoiceLine** dimension handling, internal **dict/list/dataclass** objects, and

function-level data flow, use **02_STRUCTURED_CSV_LHM_BINDINGS.md**, Chapter 15. This document remains the program specification and operating guide for the syntax-binding command.

Files

- **program_specification.md** - Defines converter inputs, outputs, dimension behavior, JSON metadata generation, reverse conversion, currency handling, XPath selector handling, and non-goals.
- **user_guide.md** - Gives command examples for forward conversion, reverse conversion, round-trip artifacts, and troubleshooting.

Related Directories

- **../src/** - Converter implementation, especially **syntax_binding.py**.
- **../specs/bindings/syntax/** - Active UBL Invoice syntax binding CSV.
- **../specs/lhm/** - LHM CSV used to define structured CSV columns and taxonomy concepts.
- **../tests/roundtrip/** - Reviewable source XML, structured CSV, metadata JSON, and regenerated XML artifacts.

Phase 1 uses EN 16931 syntax binding as the stable baseline. OpenPeppol CIUS checks are planned as a later overlay.

17. Syntax Binding User Guide

User Guide: Syntax Binding XML-to-Hierarchical-CSV Conversion

1. Working Directory

Run commands from the **UADC_PoC** directory:

```
cd UADC_PoC
```

All paths below are relative to this directory unless they begin with **../**.

Set the Python command for the local Windows environment:

```
$python = 'python'
```

2. Main Script

Use:

```
src/syntax_binding.py
```

This script converts XML to dimension-based hierarchical CSV using a syntax binding CSV. It also supports reverse conversion from hierarchical CSV back to XML with **--reverse**.

For implementation details, including XPath parent-context recursion, Semantic Path resolution, **dInvoice** and **dInvoiceLine** assignment, and function-level data flow, see **02_STRUCTURED_CSV_LHM_BINDINGS.md**, Chapter 15.

3. Convert the PoC OpenPeppol Sample

Run:

```
& $python .\src\syntax_binding.py `
.\samples\input\openpeppol_ubl_invoice_minimal.xml `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
--metadata-output .\out\phase1\openpeppol_ubl_invoice_minimal.json `
--taxonomy-base .\out\taxonomy `
-o .\out\phase1\openpeppol_ubl_invoice_minimal.csv
```

Output:

```
out/phase1/openpeppol_ubl_invoice_minimal.csv
out/phase1/openpeppol_ubl_invoice_minimal.json
```

This command uses the LHM-derived columns embedded in the syntax binding CSV to determine the output dimensions and fact columns. It also creates JSON metadata that relates the CSV columns to the generated taxonomy.

The converter treats **lhm_level** as the effective hierarchy for Structured CSV. BG rows with blank **lhm_level**, such as non-repeating Seller or Buyer groups, are semantic grouping nodes only; their BT values are written in the nearest ancestor dimension row.

4. Convert the Revised Package Sample

Run:

```
& $python .\src\syntax_binding.py `
..\syntax_binding_revised_package\invoice.xml `
-b ..\syntax_binding_revised_package\bindings.csv `
--template-csv ..\syntax_binding_revised_package\invoice.csv `
```

```
--metadata-output .\out\phase1\package_invoice_hierarchical.json `
-o .\out\phase1\package_invoice_hierarchical.csv
```

Output:

```
out/phase1/package_invoice_hierarchical.csv
out/phase1/package_invoice_hierarchical.json
```

This command uses the package template CSV to preserve the expected output column order.

5. Command Options

Basic form:

```
& $python .\src\syntax_binding.py XML_FILE `
-b BINDING_CSV `
-o OUTPUT_CSV
```

Options:

- **--template-csv**: use a template CSV header to define output column order.
- **--metadata-output**: set the JSON metadata output path. Default is the output CSV path with **.json** extension.
- **--taxonomy-base**: set the generated taxonomy directory referenced by JSON metadata. Default is **out/taxonomy**.
- **--reverse**: convert hierarchical CSV back to XML.
- **--ubl-schema-root**: local directory containing extracted UBL XSD files. In reverse mode, child element order is derived from XSD **xs:sequence** declarations.
- **--ubl-schema-url**: UBL Invoice XSD URL. In reverse mode, imported and included schemas are followed from this URL and child element order is derived from XSD **xs:sequence** declarations.
- **--drop-empty-columns**: remove output columns that have no values.
- **--d-invoice**: set the **dInvoice** value. Default is **1**.
- **-e, --encoding**: CSV encoding. Default is **utf-8-sig**.

6. Reverse Hierarchical CSV to XML

First create or confirm the hierarchical CSV:

```
& $python .\src\syntax_binding.py `
.\samples\input\openpeppol_ubl_invoice_minimal.xml `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
```

```
--metadata-output .\out\phase1\openpeppol_ubl_invoice_minimal.json `
-o .\out\phase1\openpeppol_ubl_invoice_minimal.csv
```

Then reverse it to XML:

```
& $python .\src\syntax_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
--reverse `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
-o .\out\reverse\en16931_reverse_invoice.xml
```

Output:

```
out/reverse/en16931_reverse_invoice.xml
```

To derive child element order from a local UBL schema package, add **--ubl-schema-root**:

```
& $python .\src\syntax_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
--reverse `
-b .\specs\bindings\syntax\EN16931_UBL_Invoice_Syntax_Binding.csv `
--ubl-schema-root .\out\cache\UBL-2.1\xsd `
-o .\out\reverse\en16931_reverse_invoice.xml
```

To derive child element order from an online UBL Invoice schema entry point, use **--ubl-schema-url**. The converter follows imported and included schemas from that URL.

The reverse XML is generated from bound CSV values. It is intended for round-trip verification and may not reproduce unbound XML content. When the LHM has **syntax_sequence** values, reverse output uses them to follow UBL schema order.

Amount **currencyID** attributes are derived during reverse conversion:

- **DocumentCurrencyCode** is used for ordinary invoice amount elements.
- **TaxAccountingCurrencyCode** is used for the tax accounting currency TaxTotal branch.

Forward conversion also uses those currency terms to distinguish TaxTotal branches. In **Allowance-example.xml**, BT-110 selects the TaxAmount whose **currencyID** matches **DocumentCurrencyCode** (**1225.00 EUR**), while BT-111 selects the TaxAmount whose **currencyID** matches **TaxAccountingCurrencyCode** (**9324.00 SEK**).

Some semantic children are stored outside their repeated UBL syntax context. BT-90 belongs semantically to payment instructions/direct debit, but its UBL XPath is below

AccountingSupplierParty. During reverse conversion, an absolute binding XPath that is not contained by the current repeated XPath is written from the document root. It must not produce a nested **Invoice** below **PaymentMeans**.

7. Generate Test Artifacts with Metadata

Round-trip test artifacts are built under:

```
tests/roundtrip/
```

Run:

```
& $python .\tools\build_roundtrip_test_artifacts.py
```

The generated layout is:

```
tests/roundtrip/<dataset>/  
  source_xml/  
  structured_csv/  
  metadata_json/  
  roundtrip_xml/
```

Meaning:

- **source_xml**: source UBL Invoice XML.
- **structured_csv**: hierarchical structured CSV.
- **metadata_json**: JSON metadata relating the CSV columns to the taxonomy.
- **roundtrip_xml**: XML regenerated from the structured CSV.

The script uses **--metadata-output** to place metadata JSON in **metadata_json/**.

8. Input Binding CSV

The binding CSV should contain:

```
semantic_path,xpath
```

Example:

```
$.invoice.invoiceNumber,/Invoice/cbc:ID
```

```
$.invoice.invoiceIssueDate,/Invoice/cbc:IssueDate
```

Compatible column aliases are also accepted:

```
path
source_xpath
xml_path
```

9. Output CSV Layout

The hierarchical CSV uses:

- **dInvoice** for the invoice root;
- **d*** columns for repeated BG dimensions;
- fact columns for BT values.

When the LHM is used, dimension columns are placed first and BT columns follow.

Example output pattern:

```
dInvoice,dVatBreakdown,dInvoiceLine,InvoiceNumber,InvoiceIssueDate,...
1,,,INV-2026-0001,2026-07-06,...
1,1,,,,...
1,1,,,,...
```

For a parent **dAaa** and child **dBbb**, a non-repeating child is flattened into the parent row:

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,b1V1,b2V1,b3V1
```

If **dBbb** is repeating, the parent row must leave all child facts empty, and the first child occurrence must already be on a separate row:

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

Do not put parent and repeated-child facts together on the first child row. Reverse conversion reports an error for that mixed row layout.

10. JSON Metadata Layout

The metadata file is an xBRL-CSV metadata document. It has these main sections:

- **documentInfo:** xBRL-CSV document type and taxonomy references.
- **tables:** the generated structured CSV table URL.
- **tableTemplates:** table-level dimensions and fact concept mappings.

Check that important columns are mapped as expected:

```
tableTemplates.structured.dimensions["plt:d_en16931_Invoice"] ->
"$dInvoice"
tableTemplates.structured.columns.InvoiceNumber.dimensions.concept ->
"en16931:InvoiceNumber"
tableTemplates.structured.columns.DocumentCurrencyCode.dimensions.concept
-> "en16931:DocumentCurrencyCode"
```

11. Run Regression Checks

Run the LHM-driven OpenPeppol sample conversion check:

```
& $python .\tests\test_lhm_hierarchical_csv_layout.py
```

Run the revised package conversion check:

```
& $python .\tests\test_syntax_binding.py
```

Run the OpenPeppol structured conversion check:

```
& $python .\tests\test_openpeppol_invoice_conversion.py
```

Run the reverse conversion round-trip check:

```
& $python .\tests\test_syntax_binding_reverse.py
```

Run the round-trip artifact and metadata check:

```
& $python .\tests\test_roundtrip_artifacts.py
```

Validate the generated xBRL-CSV metadata with Arelle:

```
& $python .\tests\test_xbrl_csv_metadata_arelle.py
```

Validate the regenerated round-trip XML against the UBL 2.1 Invoice schema:

```
& $python .\tests\test_roundtrip_xml_ubl_schema.py
```

12. Troubleshooting

No usable bindings found

Check that the binding CSV contains **semantic_path** and **xpath** values.

Missing output columns

If **--drop-empty-columns** is used, columns with no generated values are removed. Run without this option when a stable full header is required.

Metadata JSON cannot locate the taxonomy

The converter must write a non-empty **documentInfo.taxonomy** array. Generate the taxonomy first:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
```

Or pass **--taxonomy-base** with the directory containing **plt/en16931-oim-*.xsd**. If this schema is missing, metadata generation fails instead of writing an empty taxonomy list.

Repeating rows are missing

Check that:

- the BG row in the LHM has repeating multiplicity such as **0..*** or **1..***;
- the BG row has an XPath that points to the repeated XML context;
- the syntax binding CSV contains bindings under that BG semantic path.

Values are written to the invoice root row instead of a dimension row

Check that the syntax binding CSV contains the LHM-derived **type**, **multiplicity**, **semantic_path**, and **structured_csv_column** columns. The converter resolves each BT semantic path to its nearest

repeated C-row dimension from the binding table itself.

Package sample line rows are empty

Template columns alone do not create rows. Invoice line rows require invoice line bindings.

Reverse XML does not match the original file order

Reverse XML is reconstructed from binding rows and hierarchical CSV values. Populate LHM **syntax_sequence** from the UBL schema when XML element order must be checked. Unbound XML content still cannot be reproduced until it is represented by a binding or fixed-value rule.

Detailed Internal Walkthrough

Phase 1 Forward Conversion: UBL XML To Structured CSV

Script:

```
src/syntax_binding.py
```

Main function:

```
write_hierarchical_csv(...)
```

Input:

- UBL Invoice XML.
- Syntax binding CSV, currently
specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv.

Output:

- Phase 1 Structured CSV.
- xBRL-CSV metadata JSON.

Binding Table Interpretation

The syntax binding table contains both class rows and fact rows.

Class row:

```
type=C  
semantic_path=$.invoice.invoiceLine  
structured_csv_column=InvoiceLine
```

```

multiplicity=1..*
xpath=/Invoice/cac:InvoiceLine

```

Fact row:

```

type=A
semantic_path=$.invoice.invoiceLine.invoiceLineIdentifier
structured_csv_column=InvoiceLineIdentifier
xpath=/Invoice/cac:InvoiceLine/cbc:ID

```

The class row tells the converter where the XML context is and whether it repeats. The fact row tells the converter which value to extract inside that context.

Internal Data Structures

read_binding_rows(binding_csv, encoding) returns the raw binding table as:

```
list[dict[str, str]]
```

Each dictionary is one CSV row keyed by the header.

read_binding_layout(...) and **build_layout_from_rows(...)** build **BindingLayout**, the runtime Structured CSV layout derived from the binding table.

Important fields in **BindingLayout**:

- **fieldnames:** **list[str]**: final Structured CSV column order.
- **dimensions:** **list[str]**: dimension columns.
- **field_dimension:** **dict[str, str]**: value column to dimension column.
- **field_by_semantic_path:** **dict[str, str]**: semantic path to Structured CSV value column.
- **semantic_path_dimension:** **dict[str, str]**: semantic class path to dimension column.
- **dimension_xpath:** **dict[str, str]**: dimension column to XML context XPath.
- **dimension_ancestors:** **dict[str, list[str]]**: repeated ancestor dimensions.
- **dimension_repeats:** **dict[str, bool]**: repeat status for each emitted dimension.
- **syntax_sequence_by_field** and **syntax_sequence_by_dimension**: reverse XML ordering hints.

read_bindings(...) builds:

```
bindings: list[Binding]
```

Binding is a dataclass normalized from **type=A** rows. It stores:

- **order**;
- **semantic_path**;
- **xpath**;
- **field**, the Structured CSV value column;
- **dimension**, the row-scope dimension column;
- optional filter fields and values.

Example:

```
Binding(
    semantic_path="$.invoice.invoiceLine.invoiceLineIdentifier",
    xpath="/Invoice/cac:InvoiceLine/cbc:ID",
    field="InvoiceLineIdentifier",
    dimension="dInvoiceLine"
)
```

build_binding_class_tree(rows) builds a tree of **BindingClass** nodes from **type=C** rows.

Each **BindingClass** stores:

- **semantic_path**;
- **xpath**;
- **column**, the class column;
- **dimension**, such as **dInvoiceLine**;
- **repeats**, based on **multiplicity**;
- **children**: **list[BindingClass]**.

The class tree is the semantic and XML context tree used by the converter. It is built from the binding table, not inferred from the source XML.

direct_class_fields(class_path, bindings) groups fact bindings by their direct semantic parent. The result is used as:

```
direct_fields_by_class: dict[str, list[Binding]]
```

This prevents a parent class from extracting descendant facts too early.

Dimension Ownership Resolution

The forward and reverse paths use the same ownership metadata derived from the syntax binding table.

1. **multiplicity_repeats(multiplicity)** recognizes *****, **n**, **unbounded**, or a numeric upper bound greater than 1 as repeating.

2. **build_layout_from_rows(rows)** reads every **type=C** class row. The invoice root becomes **dInvoice**. Other class rows become **dXxx** only when their multiplicity repeats.
3. While reading each **type=A** row, the nested **nearest_dimension(...)** helper walks upward through its **semantic_path** until it finds the nearest class that owns a dimension.
4. The result is stored in **BindingLayout.field_dimension** and copied into each normalized **Binding.dimension** by **read_bindings(...)**.

Therefore a non-repeating child class belongs to its nearest repeating ancestor row. For example, non-repeating item information below a repeating invoice line belongs to **dInvoiceLine**. A repeating child class owns its own dimension and is not merged into its parent row.

The ownership rule is independent of the target XML QName. It is determined by **type=C**, **multiplicity**, and **semantic_path**, while **structured_csv_column** supplies the actual fact and class column names.

XPath Context Recursion

The forward conversion uses XML parent-context recursion. The central nested function inside **write_hierarchical_csv(...)** is:

```
process_class(context, class_node, dimension_values, current_row)
```

Arguments:

- **context: ET.Element**: XML element for the current class.
- **class_node: BindingClass**: semantic class being processed.
- **dimension_values: dict[str, str]**: active dimension values inherited from parent contexts.
- **current_row: dict[str, str] | None**: current Structured CSV row.

At the root, the converter calls:

```
process_class(root_context, class_root, {"dInvoice": "1"}, None)
```

This means the first invoice has **dInvoice=1**.

How A Rows Are Filled

For each class context, **fill_direct_fields(row, context, class_node)** processes only the direct **A** rows for that class.

For each **Binding**:

1. **relative_xpath(binding.xpath, class_node.xpath)** converts the absolute binding XPath into a path relative to the current class context. If the binding points outside that context, the path remains absolute and forward extraction evaluates it from the document root.

2. **get_value(context, relative_xpath, namespaces, root)** extracts text or attribute value.
3. The value is written to **row[binding.field]**.

Example:

Class:

```
$.invoice.invoiceline
/Invoice/cac:InvoiceLine
```

Fact:

```
$.invoice.invoiceline.invoiceLineIdentifier
/Invoice/cac:InvoiceLine/cbc:ID
```

Relative XPath:

```
cbc:ID
```

The converter extracts **cbc:ID** from the current **cac:InvoiceLine** context and writes it to **InvoiceLineIdentifier**.

How Repeated Dimensions Are Decided

Repeated dimensions are decided from **BindingClass.repeats**, which is derived from the class row **multiplicity**.

When **process_class(...)** sees a child class:

- if the child is non-repeated, it processes that child using the same **current_row**;
- if the child is repeated and has a dimension, it delays that child into **repeated_children**.

After the current row is filled and written if needed, repeated children are processed one occurrence at a time.

For each repeated child occurrence:

1. **class_contexts(parent_context, parent_class, child_class)** finds child XML contexts under the current parent XML context.
2. The occurrence number starts at 1.
3. A new **child_dimensions** dictionary is created by copying parent dimensions.
4. The child dimension is set.
5. A new row is created.
6. The function recurses into the child XML context.

Example:

```
Parent dimensions:
{
  "dInvoice": "1"
}

Second invoice line:
{
  "dInvoice": "1",
  "dInvoiceLine": "2"
}

First item attribute under that line:
{
  "dInvoice": "1",
  "dInvoiceLine": "2",
  "dItemAttributes": "1"
}
```

This is how **dInvoice**, **dInvoiceLine**, and deeper dimension values are determined. The value is the 1-based occurrence number inside the current XML parent context.

Row Creation And Sparse Rows

Rows are dictionaries:

```
dict[str, str]
```

new_row(fieldnames, d_invoice) creates a row with all columns initialized to blank and **dInvoice** initialized.

row_has_values(row, dimension_columns) prevents dimension-only empty rows from being appended.

The final output is:

```
rows: list[dict[str, str]]
```

This is written by **csv.DictWriter** using **fieldnames** from **BindingLayout**.

For a repeating child, **process_class(...)** completes and appends the parent row before processing **repeated_children**. It then calls **new_row(...)** for every child occurrence, including occurrence 1. Consequently the first child cannot share the parent row:

```
dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2
```

When **dBbb** is non-repeating, **process_class(...)** passes the same **current_row** to the child and the result is instead:

```
1,,a1V1,a2V1,b1V1,b2V1,b3V1
```

Phase 1 Reverse Conversion: Structured CSV To UBL XML

Script:

```
src/syntax_binding.py --reverse
```

Main function:

```
write_xml_from_hierarchical_csv(...)
```

Input:

- Structured CSV.
- Same UBL syntax binding table.

Output:

- UBL Invoice XML.

Reverse Data Interpretation

Reverse mode reads:

```
rows: list[dict[str, str]]
bindings: list[Binding]
```

Each Structured CSV row is sparse. Dimension columns determine where the row belongs in the XML hierarchy.

Example:

```
dInvoice=1
dInvoiceLine=2
InvoiceLineNetAmount=1000
```

This means the value belongs under the second **cac:InvoiceLine**.

Before XML construction, **validate_hierarchical_row_scopes(rows, fieldnames, field_dimension)** applies the ownership metadata built by **build_layout_from_rows(...)**:

1. It identifies dimension columns with **is_dimension_column(...)**.
2. For each input row, it takes the deepest populated dimension as the row scope.
3. For each populated fact, it looks up the owner in **BindingLayout.field_dimension**.
4. It raises **ValueError** when the owner differs from the row scope.

This rejects a mixed row such as **1,1,a1V1,a2V1,b1V1,b2V1,b3V1**. The parent facts must remain on the parent row and the repeating child facts must start on a separate child row.

XML Construction

The reverse writer creates:

```
root: ET.Element
```

It then writes values using binding XPaths.

Important helper functions:

- **ensure_path(root, xpath, namespaces, force_new_leaf=False)**: walks an XPath and creates missing XML elements.
- **find_or_create_child(parent, step, namespaces, force_new=False)**: finds a child element matching a tag and predicate or creates it.
- **create_context(...)**: creates or reuses the XML parent context for a repeated row.
- **set_xml_value_with_currency(...)**: writes a value and applies **currencyID** when the target is an amount.
- **set_relative_xml_value(...)**: writes paths contained by the current repeated syntax context.
- An absolute XPath that remains outside the repeated context is passed to **set_xml_value_with_currency(...)** with the document root. This covers cross-scope mappings such as BT-90: its semantic path is below payment instructions, while its UBL XPath is below **AccountingSupplierParty**.
- **set_relative_xml_value(...)**: writes a value below a repeated context.
- **ensure_tax_scheme_defaults(...)**: adds required UBL tax scheme defaults.
- **load_ubl_child_order(...)**: reads UBL XSD files from **--ubl-schema-root** or **--ubl-schema-url** and builds child element order from **xs:sequence**.

- **sort_children_for_ubl_schema(...)**: reorders children using generated UBL schema order. If no schema source is supplied, it uses the built-in fallback order for the current PoC structures.

The reverse process is path-construction based. It uses the same binding table, but currently does not use the same recursive **BindingClass** tree as forward mode. Repeated-context paths are made relative only when the binding XPath is contained by the repeated syntax path; out-of-context absolute paths remain document-rooted. It remains schema-order aware and can use UBL XSD order for UBL 2.1 through later UBL versions when a schema root or schema URL is supplied.